

Chapter 6 Decision Trees

Austin R.J. Downey

Decision Trees

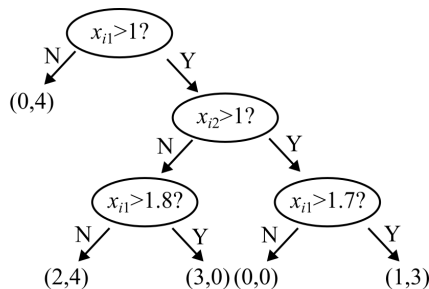
Decision Trees are flexible models used for:

- ▶ Classification
- ▶ Regression
- ▶ Multi-output prediction

Key ideas:

- ▶ Split data using simple rules
- ▶ Build a tree of decisions
- ▶ Make predictions by following a path

They are also the building blocks of Random Forests.



Basic concept of a decision tree.

Decision Trees

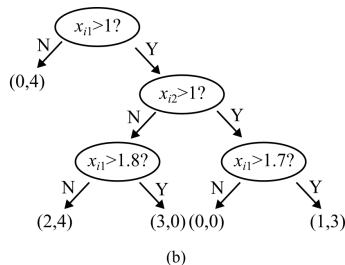
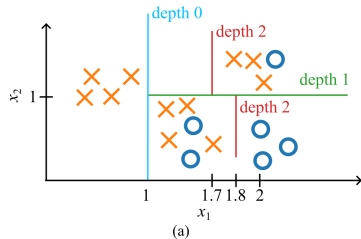
Decision Trees are flexible models used for:

- Classification
- Regression
- Multi-output prediction

Key ideas:

- Split data using simple rules
- Build a tree of decisions
- Make predictions by following a path

They are also the building blocks of Random Forests.



Basic concept of a decision tree.

Decision Tree Classification

A decision tree partitions the feature space into regions.

For the Iris dataset:

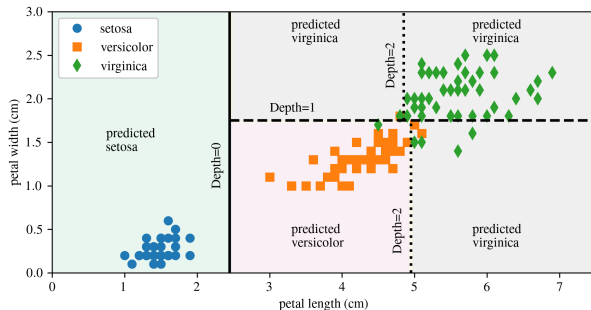
► Root split:

$$\text{petal length} \leq 2.45$$

► Second split:

$$\text{petal width} \leq 1.75$$

Each region is assigned a predicted class.



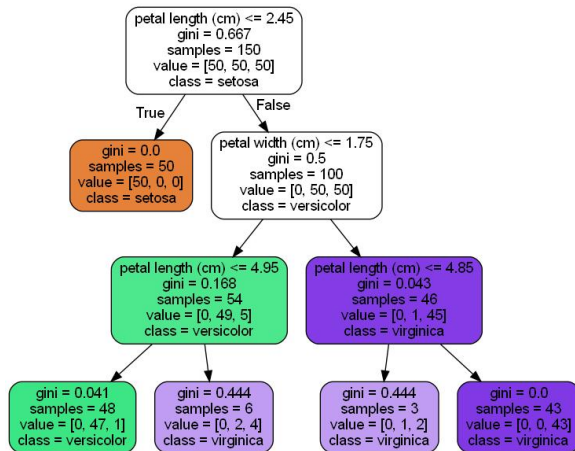
Decision tree decision boundaries.

Reading a Decision Tree

To classify a new sample:

1. Start at the root node
2. Answer the split question
3. Move left or right
4. Continue until a leaf node is reached

The leaf node gives the predicted class.



Decision tree trained on the Iris dataset.

Node Information

Each node in a classification tree stores useful information.

- ▶ **class**: predicted class at the node
- ▶ **samples**: number of training instances that reach the node
- ▶ **value**: class counts at the node
- ▶ **gini**: impurity of the node

A node is pure when all samples belong to the same class:

$$G_i = 0$$

Darker node colors indicate purer predictions.

Gini Impurity

Gini impurity measures how mixed the classes are at a node.

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

where:

- ▶ $p_{i,k}$ is the proportion of class k at node i
- ▶ $G_i = 0$ means the node is pure
- ▶ Larger values indicate more mixed classes

Example:

$$1 - \left(\frac{0}{54}\right)^2 - \left(\frac{49}{54}\right)^2 - \left(\frac{5}{54}\right)^2 \approx 0.168$$

Class Probability Estimation

Decision Trees can estimate class probabilities.

Procedure:

1. Follow the sample to a leaf node
2. Count the training samples in that leaf
3. Compute the fraction belonging to each class

Example leaf node:

$$\text{value} = [0, 49, 5]$$

Probabilities:

$$P(\text{setosa}) = 0\%, P(\text{versicolor}) = \frac{49}{54} = 90.7\%, \text{ and } P(\text{virginica}) = \frac{5}{54} = 9.3\%$$

Example 6.1: Decision Tree Classifier

- ▶ Use `DecisionTreeClassifier`
- ▶ Classify Iris species using petal features
- ▶ Train a tree with a limited depth
- ▶ Visualize the learned decision rules with Graphviz

The exported tree shows:

- ▶ Feature thresholds
- ▶ Gini impurity
- ▶ Sample counts
- ▶ Class distributions

Graphviz viewer:

<https://dreampuf.github.io/GraphvizOnline/>

The CART Training Algorithm

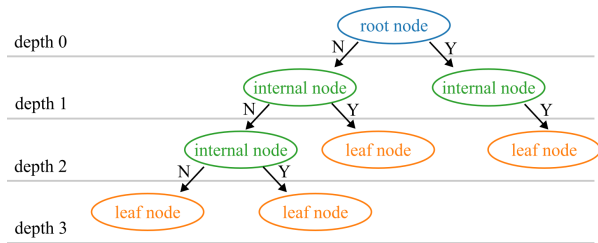
Scikit-Learn trains decision trees
using CART:

Classification and Regression Trees

At each node, CART chooses:

- ▶ A feature k
- ▶ A threshold t_k

The goal is to create child nodes
that are as pure as possible.



CART recursively grows the tree.

CART Objective

At each split, CART minimizes:

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

where:

- ▶ G_{left} and G_{right} are impurity values
- ▶ m_{left} and m_{right} are sample counts
- ▶ m is the total number of samples at the current node

The tree grows recursively until:

- ▶ A maximum depth is reached
- ▶ No useful impurity-reducing split remains

Regularizing Decision Trees

Decision Trees can easily overfit.

Regularization limits how much freedom the tree has during training.

Important Scikit-Learn hyperparameters:

- ▶ `max_depth`
- ▶ `min_samples_split`
- ▶ `min_samples_leaf`
- ▶ `max_leaf_nodes`
- ▶ `max_features`

General rule:

- ▶ Decrease `max_*` values
- ▶ Increase `min_*` values

Regularization Effects

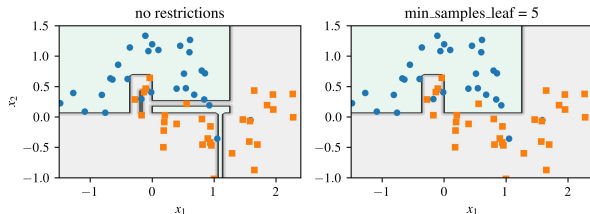
An unrestricted tree may fit noise in the training data.

Adding restrictions can improve generalization.

Example:

`min_samples_leaf = 4`

This prevents leaves from being created from very small groups of samples.



Regularization using `min_samples_leaf`.

Computational Complexity

Prediction is fast.

A prediction follows one path from the root to a leaf:

$$T_{\text{predict}} = O(\log_2 m)$$

Training is more expensive:

$$T_{\text{train}} = O(nm \log m)$$

where:

- ▶ m : number of training instances
- ▶ n : number of input features

Decision Trees are especially efficient at prediction time.

Gini Impurity vs. Entropy

Scikit-Learn uses Gini impurity by default.

Entropy is another impurity measure:

$$H_i = - \sum_{k=1}^n p_{i,k} \log_2(p_{i,k})$$

Comparison:

- ▶ Gini is slightly faster to compute
- ▶ Gini often isolates the most frequent class
- ▶ Entropy tends to produce more balanced trees

In practice, both usually produce similar trees.

Decision Tree Regression

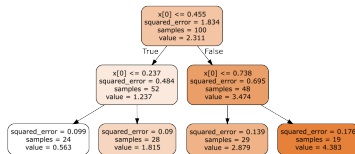
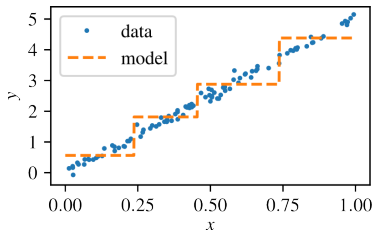
Decision Trees can also perform regression.

Instead of predicting a class, each leaf predicts:

$$\hat{y}_{\text{node}}$$

This value is the average target value of the training instances in that leaf.

The model creates piecewise-constant predictions.



Decision Tree regression model and tree.

Tree Depth in Regression

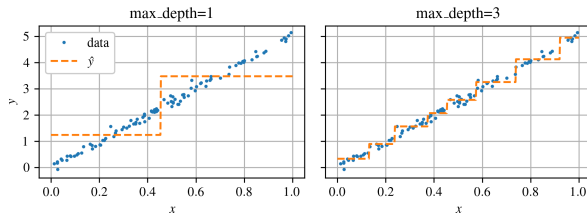
Increasing tree depth creates more prediction regions.

Shallow tree:

- ▶ Fewer regions
- ▶ Simpler model
- ▶ More bias

Deeper tree:

- ▶ More regions
- ▶ More flexible model
- ▶ Higher risk of overfitting



Regression trees with different depths.

Regression Tree Cost Function

For regression, CART minimizes MSE instead of impurity.

$$J(k, t_k) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}}$$

At each node:

$$\text{MSE}_{\text{node}} = \sum_{i \in \text{node}} \left(\hat{y}_{\text{node}} - y^{(i)} \right)^2$$

$$\hat{y}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} y^{(i)}$$

The tree chooses splits that reduce prediction error.

Regularizing Regression Trees

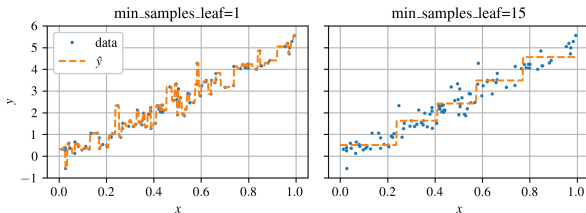
Regression trees can also overfit.

Without regularization:

- ▶ Tree follows noise
- ▶ Predictions become jagged

With regularization:

- ▶ Leaves contain more samples
- ▶ Predictions are smoother
- ▶ Generalization improves



Effect of `min_samples_leaf` on regression trees.

Example 6.2: Decision Tree Regression

- ▶ Use `DecisionTreeRegressor`
- ▶ Fit a nonlinear relationship between input and output data
- ▶ Train a tree with limited depth
- ▶ Visualize the regression tree using Graphviz

This example shows how a tree partitions the input space into regions. Each region predicts the average target value of the samples it contains.

Instability: Feature Orientation

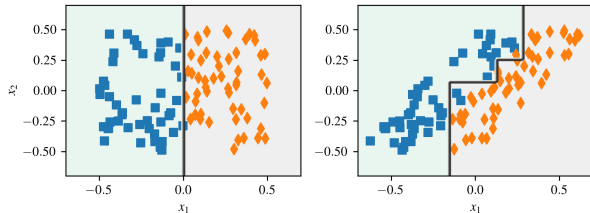
Decision Trees prefer orthogonal splits.

This makes them sensitive to feature orientation.

A simple rotation of the data can produce:

- ▶ More complex boundaries
- ▶ Poorer generalization
- ▶ Less stable predictions

PCA can sometimes help reorient the data.



Sensitivity to feature rotation.

Instability: Training Variation

Decision Trees can change significantly with small training variations.

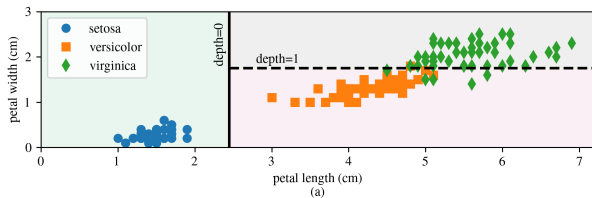
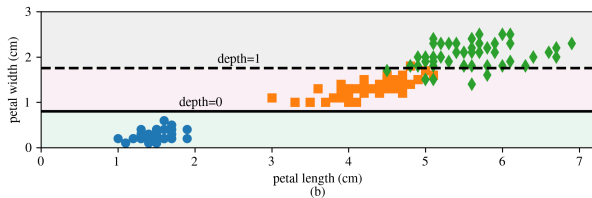
Sources of variation:

- ▶ Random number seed
- ▶ Equal-quality candidate splits
- ▶ Small changes in training data

To improve reproducibility:

`random_state`

can be fixed during training.



Sensitivity to initial conditions.

Random Forests

A Random Forest builds many Decision Trees.

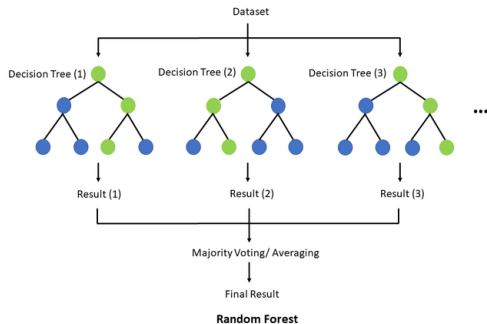
Each tree is trained using:

- ▶ A bootstrap sample of the data
- ▶ A random subset of features

Predictions are combined by:

- ▶ Majority vote for classification
- ▶ Averaging for regression

This reduces variance compared with a single tree.



Random forest with majority voting.